GENERALITAT VALENCIANA
CONSELLERIA D'EDUCACIÓ, INVESTIGACIÓ, CULTURA I ESPORT

IES JUAN DE GARAY
C /Juan de Garay, 25 46017 VALENCIA
Tel.: 961206045 Fax: 961206046
46012963@gva.es juandegaray.com

Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

Taller 3

Sistema RAG Estándar

Retrieval-Augmented Generation con n8n

Asistente legal inteligente que consulta documentos PDF internos mediante búsqueda semántica por similitud vectorial

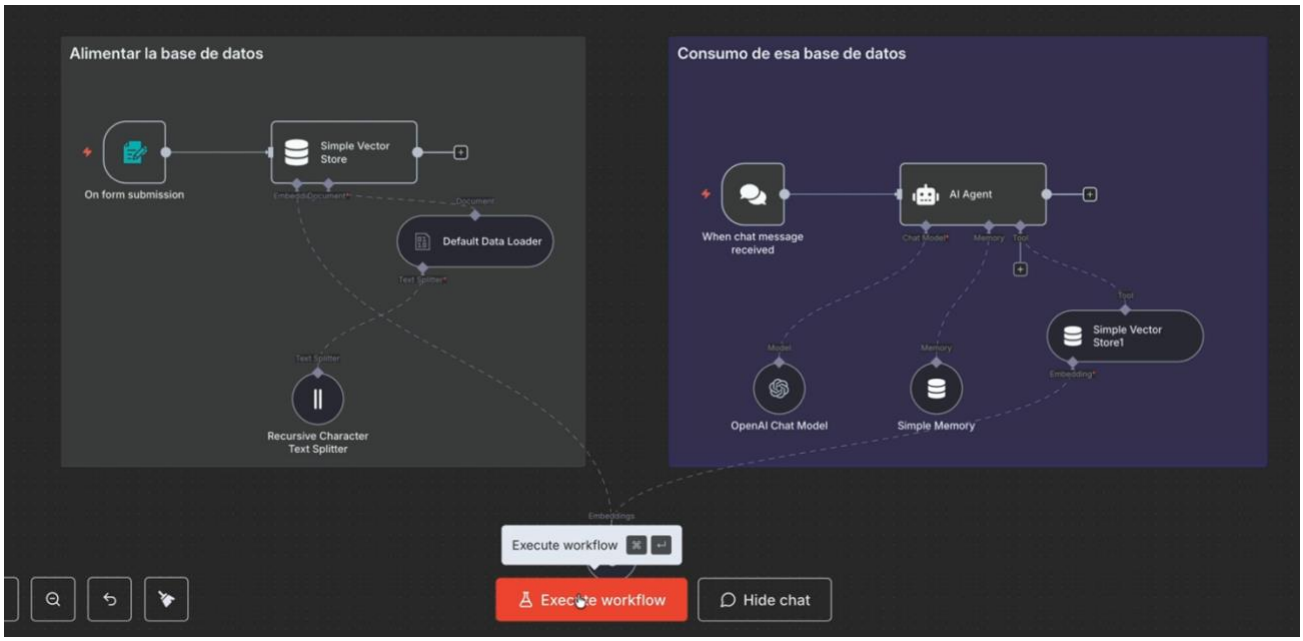
¿Qué es un sistema RAG? *3*
Fase 1: Nodos de Ingestión *5*
Nodo 1: [INGESTION] On form submission (Form Trigger) **5**
Nodo 5: [INGESTION] Simple Vector Store (modo insert) **7**
Nodo 2: [INGESTION] Default Data Loader (Cargador de documentos) **10**
Nodo 3: [INGESTION] Recursive Character Text Splitter **11**
Nodo 4: [COMPARTIDO] Embeddings Ollama **12**
Fase 2: Nodos de Consumo (RAG) *13*
Nodo 6: [CONSUMO] When chat message received (Chat Trigger) **13**
Nodo 8: [CONSUMO] Ollama **16**
Nodo 9: [CONSUMO] Simple Memory (Memoria de conversación) **17**
Nodo 10: [CONSUMO] Simple Vector Store1 (modo retrieve-as-tool) **18**
System Prompt del Asistente Legal *19*
Comparativa con los otros workflows *20*

¿Qué es un sistema RAG?

RAG (Retrieval-Augmented Generation) es una arquitectura de IA que combina la búsqueda semántica con la generación de texto. En lugar de responder solo desde el conocimiento pre-entrenado del modelo, el sistema primero recupera fragmentos relevantes de una base de documentos propia, y luego usa esos fragmentos como contexto para generar la respuesta.

Este workflow implementa un RAG estándar de dos fases bien diferenciadas, tal como se refleja en el Canvas de n8n con dos secciones (sticky notes):

FASE 1: Alimentar la base de datos	FASE 2: Consumo de la base de datos
Se cargan los PDFs, se fragmentan en chunks, se vectorizan con embeddings y se guardan en el vector store. Se ejecuta una vez (o cada vez que hay documentos nuevos).	El abogado hace una pregunta por chat. El agente vectoriza la pregunta, busca chunks similares en el store, los usa como contexto y genera una respuesta con referencia al documento.



Arquitectura del sistema

El workflow tiene 10 nodos organizados en dos pipelines paralelos que comparten un nodo critico: el modelo de embeddings.

FASE 1 - Pipeline de Ingestión

Form Trigger > Data Loader + Splitter > Embeddings OpenAI > Vector Store (insert)

FASE 2 - Pipeline de Consumo (RAG)

Chat Trigger > AI Agent + GPT-4.1 + Memory > Embeddings OpenAI > VS1 > Respuesta

Flujo de datos detallado

Durante la ingestión (cargar un PDF)

1	El usuario sube un PDF vía el formulario web (Form Trigger)
2	Default Data Loader extrae el texto del PDF en binario
3	Recursive Character Text Splitter divide el texto en chunks de ~1000 caracteres con 250 de overlap
4	Embeddings OpenAI (text-embedding-3-large) convierte cada chunk en un vector de 3072 dimensiones
5	Simple Vector Store (insert) guarda los pares (chunk_texto, vector) bajo la clave vector_store_key

Durante el consumo (pregunta del abogado)

1	El abogado escribe una pregunta en el chat (Chat Trigger)
2	El AI Agent recibe la pregunta y decide usar la tool del Vector Store
3	Embeddings OpenAI vectoriza la pregunta del abogado (mismo modelo que en ingestión)
4	Simple Vector Store1 busca los N chunks con mayor similitud coseno al vector de la pregunta
5	Los chunks recuperados se inyectan como contexto en el prompt del LLM
6	El modelo genera una respuesta concisa con referencia al documento fuente
7	Simple Memory guarda el par pregunta-respuesta para mantener contexto en el siguiente turno

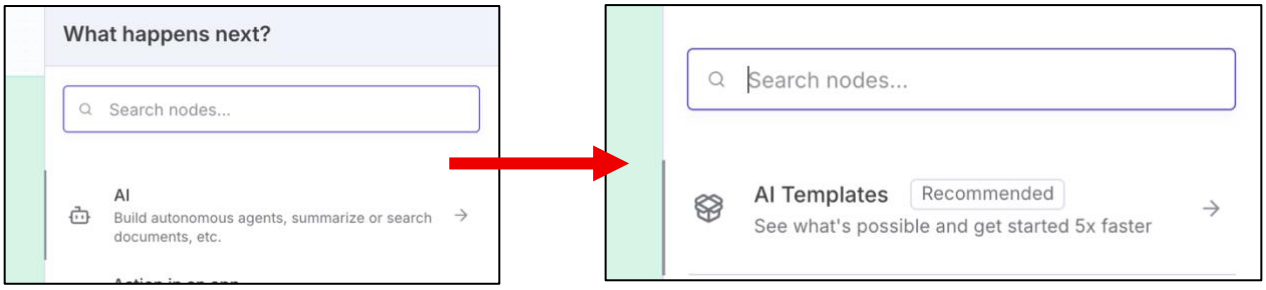
Fase 1: Nodos de Ingestión

Estos 5 nodos forman el pipeline de carga de documentos. Se ejecutan una vez por cada lote de PDFs que el usuario sube al formulario.

Podríamos partir de plantillas ya predefinidas (de hecho ahorra mucho tiempo), podemos llegar directamente por el enlace:

https://n8n.io/workflows/categories/ai/?utm_source=n8n_app&utm_medium=template_library&utm_instance=https://vps-86c1c2aa.vps.ovh.net/&utm_n8n_version=1.120.4&utm_awc=0&utm_user_role=AdvancedAI

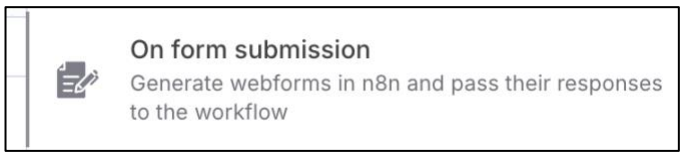
O por el menú:



Pero con el objetivo de aprender los iremos añadiendo uno a uno los nodos necesarios.

Nodo 1: [INGESTION] On form submission (Form Trigger)

Es el punto de entrada de la fase de ingestión. Expone un formulario web publico donde el usuario puede subir uno o varios archivos PDF. Al enviar el formulario, los archivos binarios se pasan al pipeline de procesamiento. Este nodo es la única forma de cargar documentos al vector store en este workflow.



Valor / Configuración

Tipo de nodo

n8n-nodes-base.formTrigger

Form title

Cargar documentos

Campo: Documentos

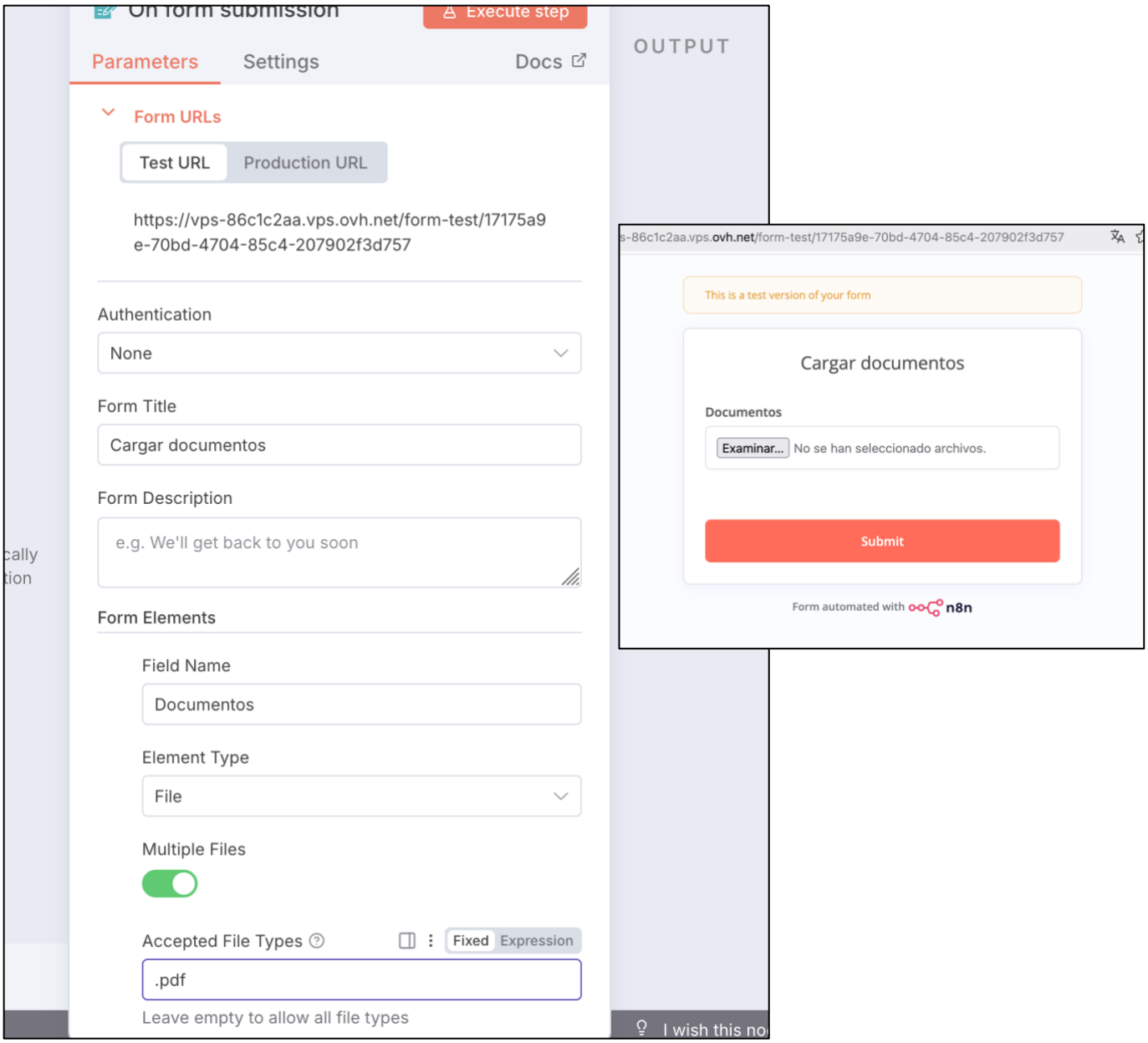
Tipo file, acepta solo .pdf

Destino

Simple Vector Store (insert)

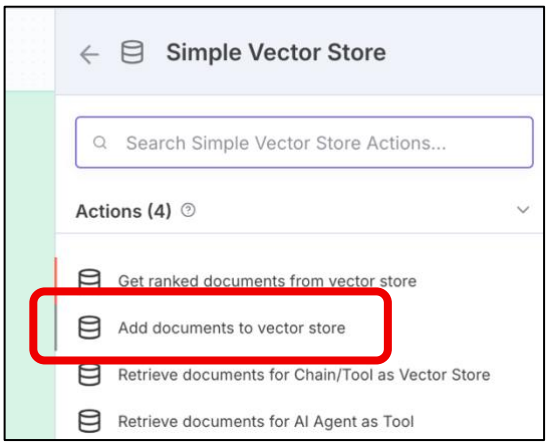
Consejo: Puedes ampliar los tipos de archivo aceptados cambiando acceptFileTypes a '.pdf,.docx,.txt'. También puedes reemplazar este trigger por un Google Drive Trigger para cargar documentos automáticamente cuando se suban a una carpeta.

El nodo se debería ver de la siguiente manera:



Nodo 5: [INGESTION] Simple Vector Store (modo insert)

Almacena los embeddings generados en un vector store en memoria. Esta instancia opera en modo 'insert': recibe los chunks vectorizados y los guarda bajo la clave 'vector_store_key'. El vector store en memoria es volátil: los datos se pierden cuando n8n se reinicia. Es ideal para demos y desarrollo.



Campo

Valor / configuración

Tipo de nodo

@n8n/n8n-nodes-langchain.vectorStoreInMemory

Mode

insert (escritura de embeddings)

Memory key

vector_store_key

Embedding batch size

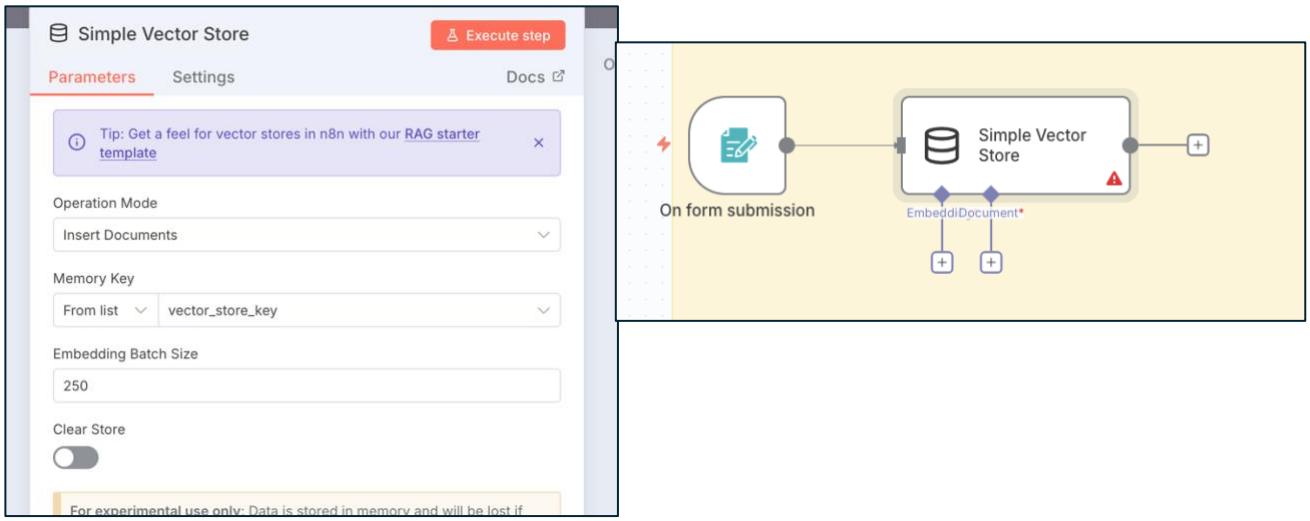
250 chunks por lote

Fuente de embeddings

Embeddings OpenAI

Consejo: El vector store en memoria es volátil: se vacía al reiniciar n8n. Para producción, reemplázalo por Pinecone, Qdrant, Supabase (pgvector) o cualquier vector DB persistente. La memory key 'vector_store_key' debe coincidir exactamente entre esta instancia (insert) y la instancia de consumo (retrieve).

Inicialmente quedará así:



Ahora deberemos configurar:

<u>Embeddings</u>: Los embeddings son como convertir cada documento en una coordenada en un mapa.

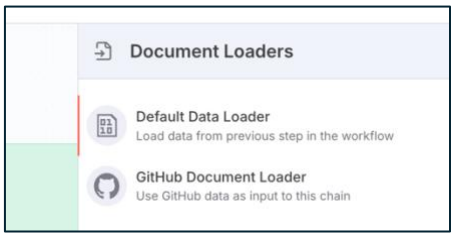
Imagina que cada texto tiene su propio punto en un mapa gigante:

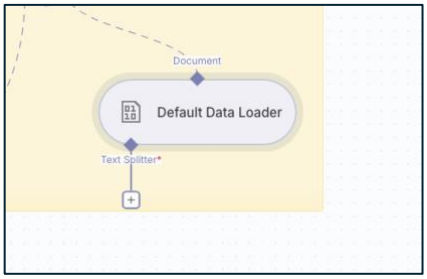
- Todos los textos sobre **animales** están en el norte del mapa
- Todos los textos sobre **leyes y contratos** están en el este
- Todos los textos sobre **cocina** están en el sur

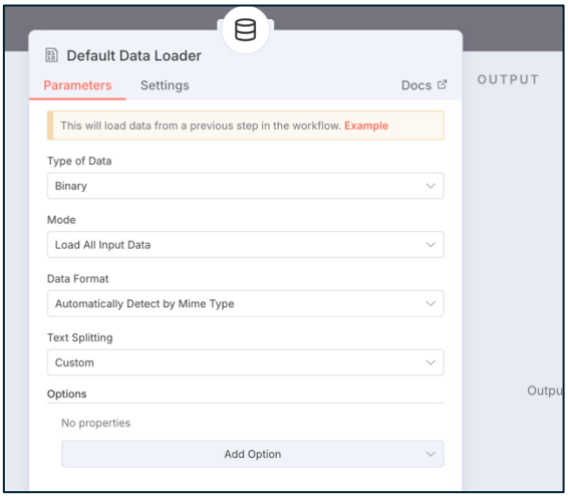
Cuando tú escribes una pregunta, el sistema la convierte también en un punto del mapa, y luego simplemente busca **qué otros puntos están más cerca** de donde estás tú.

En nuestro caso usaremos Ollama que es el que tenemos en local.

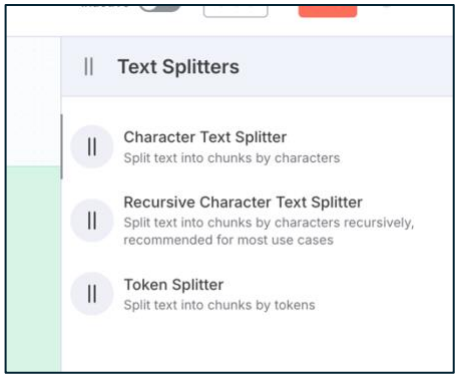
Document Loader: es el que nos cargará los archivos como queramos.

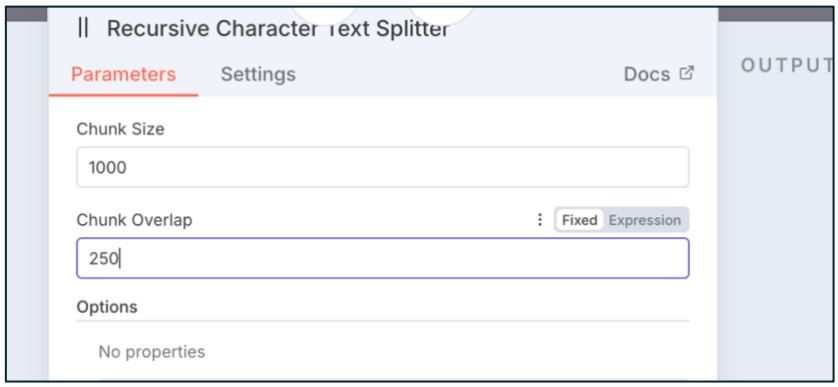






Ahora configuraremos el Text Splitter:





- **Chunk size**: los bloques en que se divide el texto.

- **Chunk overlap**: los bloques que se solapan entre chunks, sirven para organizar la información y saber que bloques van antes.

Nodo 2: [INGESTION] Default Data Loader (Cargador de documentos)

Recibe los archivos binarios del formulario y los convierte en documentos de texto que el pipeline de embeddings puede procesar. Trabaja en modo 'binary data' y usa el Text Splitter conectado para fragmentar el contenido antes de enviarlo al vector store.

Valor / Configuracion

Tipo de nodo

@n8n/n8n-nodes-langchain.documentDefaultDataLoader

Data type

binary (archivos subidos directamente)

Text splitting mode

custom (usa el splitter conectado)

Text Splitter conectado

Recursive Character Text Splitter

Destino

Simple Vector Store (ai_document)

Consejo: Este nodo soporta PDF, DOCX, TXT y otros formatos. El modo 'custom' permite usar cualquier splitter que conectes como sub-nodo, lo que da control total sobre como se fragmentan los documentos.

Nodo 3: [INGESTION] Recursive Character Text Splitter

Divide el texto extraído de los documentos en fragmentos (chunks) de tamaño manejable para generar embeddings. Usa una estrategia recursiva que intenta respetar los límites naturales del texto (párrafos, oraciones, palabras) antes de cortar. El chunk overlap garantiza continuidad de contexto entre fragmentos.

Valor / Cconfiguración

Tipo de nodo

@n8n/n8n-nodes-langchain.textSplitterRecursiveCharacterTextSplitter

Chunk overlap

250 caracteres de solapamiento entre chunks

Chunk size

Por defecto (1000 caracteres)

Separadores

Recursivos: párrafo, línea, espacio, carácter

Destino

Default Data Loader (ai_textSplitter)

Consejo: El overlap de 250 garantiza que el contexto no se corte bruscamente en el límite de dos chunks. Si tus documentos tienen párrafos muy largos, aumenta el chunk size a 1500-2000. Para documentos con mucha estructura (contratos, clausulas), considera un chunk size menor como 500.

Nodo 4: [COMPARTIDO] Embeddings Ollama

Convierte los chunks de texto en vectores numéricos (embeddings) de alta dimensión. Este nodo es compartido entre ambas fases: en la ingestión vectoriza los chunks antes de guardarlos, y en el consumo vectoriza la pregunta del usuario para buscar chunks similares por proximidad geométrica. Usa el modelo Ollama.

Valor / configuración

Credenciales

Ollama

Conectado a

Simple Vector Store (insert) + Simple Vector Store1 (retrieve)

Options

Por defecto

Consejo: el mismo modelo debe usarse en ingestión y en consulta.

Fase 2: Nodos de Consumo (RAG)

Estos 5 nodos forman el pipeline de consulta. Se activan con cada mensaje del abogado en el chat y ejecutan el ciclo completo: recuperación semántica, aumentación y generación de respuesta.

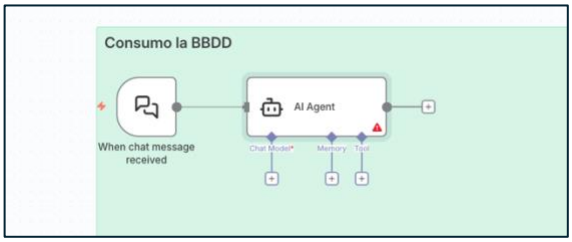
Nodo 6: [CONSUMO] When chat message received (Chat Trigger)

Punto de entrada de la fase de consumo. Expone un webhook de chat que recibe las preguntas del abogado. A diferencia del trigger del Wikipedia Agente, este opera sin acceso público (solo interno) y sus opciones están por defecto. Cada mensaje recibido activa el AI Agent con el contenido del chat (aunque lo podemos hacer público).

Campo
Valor / configuración
Tipo de nodo
@n8n/n8n-nodes-langchain.chatTrigger
Options
Por defecto (sin acceso público)
Webhook ID
1ee80bac-064b-4a80-912e-8fcd9041ed40
Destino
AI Agent (main input)
Consejo: Al no tener public: true, este chat solo es accesible desde la interfaz de n8n. Para exponerlo al exterior (como se hace con index.html en el Wikipedia Agente), activa la opción public en el nodo y conecta un frontend con @n8n/chat.

Nodo 7: [CONSUMO] AI Agent (Asistente Legal)

El núcleo inteligente del sistema RAG. Recibe la pregunta del abogado, usa el Vector Store como herramienta de recuperación para encontrar los chunks más relevantes, los combina con el system prompt legal, y genera una respuesta concisa con referencia al documento fuente. Implementa el patrón RAG completo: Retrieve, Augment, Generate.



Valor / Configuración

Tipo de nodo

@n8n/n8n-nodes-langchain.agent

Rol (system prompt)

Asistente Legal - respuestas cortas con referencias

LLM conectado

Ollama

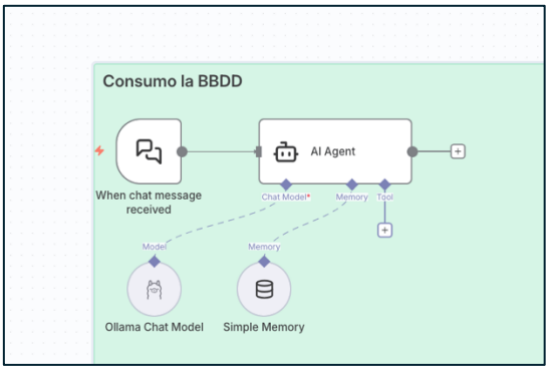
Memoria conectada

Simple Memory (10 mensajes)

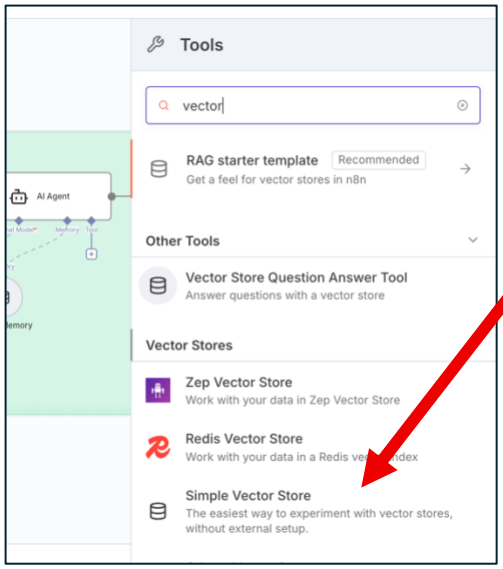
Tool conectada

Simple Vector Store1 (retrieve-as-tool)

Consejo: El system prompt instruye al agente a citar siempre el documento fuente (nombre de archivo y clausula). Esto es crítico en contextos legales para que el abogado pueda verificar la información directamente en el documento original.

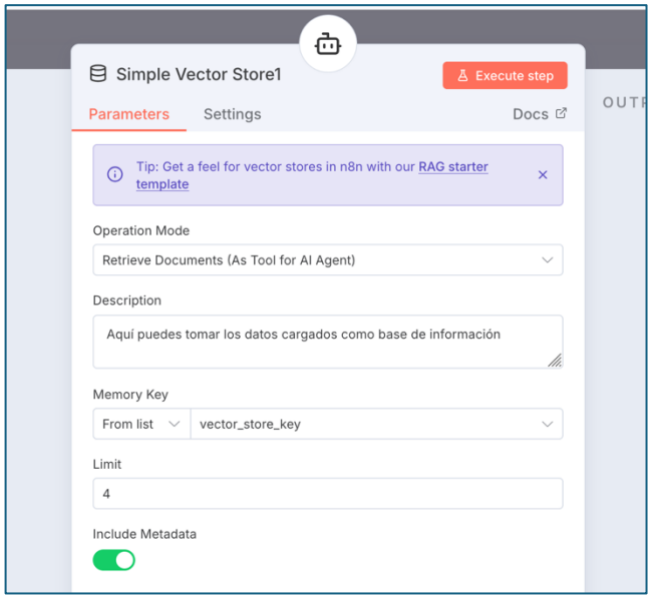


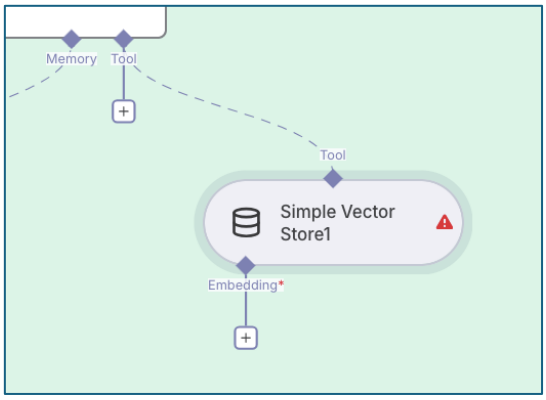
Tal y como hicimos en el taller anterior conectaremos a Ollama el modelo y la memoria la haremos simple y que se guarde sólo los 10 últimos mensajes.



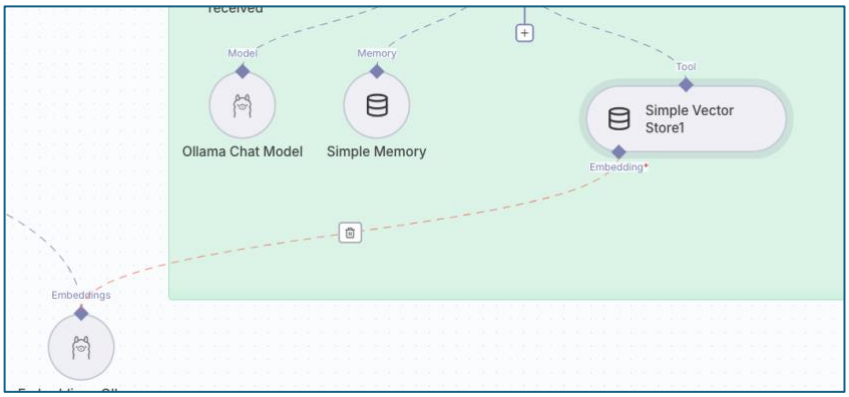
Como tool añadiremos una que gestione la consulta como es el Simple Vector Store.

Y las opciones de este tool:





Ahora sólo nos falta la opción Embedding enlazarla con el Embedding donde cargábamos los datos para que los lea de allí.



Nodo 8: [CONSUMO] Ollama

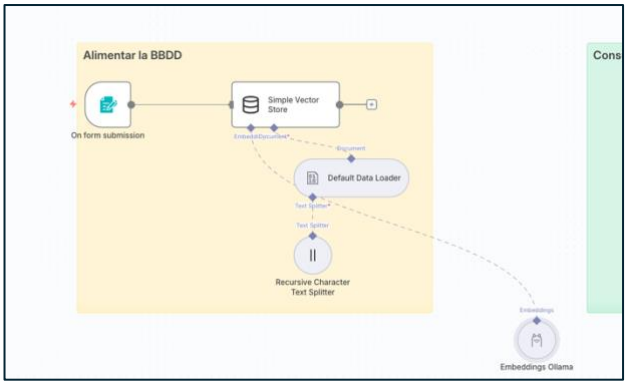
Modelo de lenguaje que genera las respuestas del asistente legal. Ollama ofrece un buen y recibe como contexto los chunks recuperados del vector store junto con el historial de la conversación.

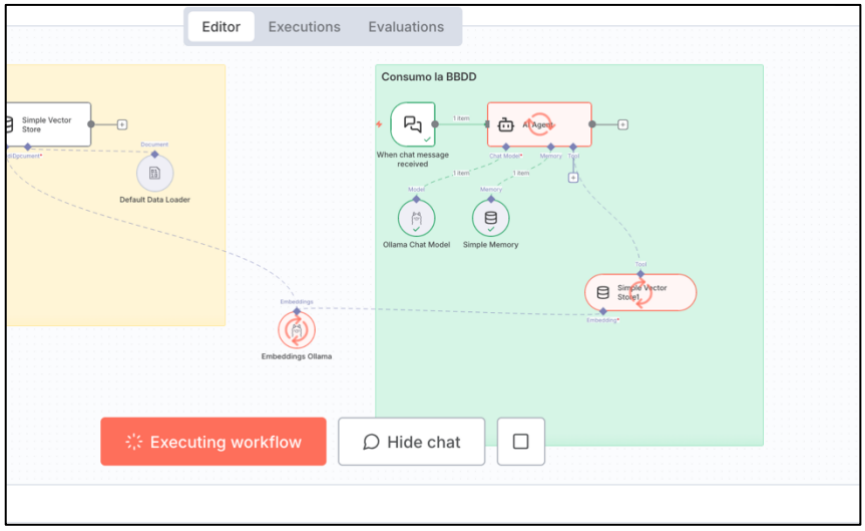
Valor / Configuración

Modelo

Ollama

Y ahora ya podemos cargar los archivos a consultar.





Importante en el modelo embedding es usar Ollama 3.1, los modelos posteriores dan error y no gestionan bien el Embedding.

Nodo 9: [CONSUMO] Simple Memory (Memoria de conversación)

Mantiene el historial de los últimos 10 mensajes de la conversación (5 preguntas + 5 respuestas). ¿Permite al agente responder preguntas de seguimiento como 'y en que clausula aparece?' sin perder el contexto de la pregunta anterior. Es memoria volátil: se borra al reiniciar n8n.

Valor / configuración

Tipo de nodo

@n8n/n8n-nodes-langchain.memoryBufferWindow

Context window length

10 mensajes

Persistencia

En memoria (volátil, se borra con n8n)

Destino

AI Agent (ai_memory)

Consejo: Para un sistema legal en produccion, la memoria debe ser persistente entre sesiones. Reemplaza Simple Memory por un Memory Manager con Redis o PostgreSQL para que el abogado pueda retomar conversaciones anteriores.

Nodo 10: [CONSUMO] Simple Vector Store1 (modo retrieve-as-tool)

Segunda instancia del vector store, operando en modo 'retrieve-as-tool'. A diferencia del primero (que escribe), este lee: vectoriza la pregunta del usuario, busca los N chunks más similares por distancia coseno, y los devuelve al agente como contexto. Usa la misma clave 'vector_store_key' para acceder a los documentos cargados.

Valor / Configuración

Tipo de nodo

@n8n/n8n-nodes-langchain.vectorStoreInMemory

Mode

retrieve-as-tool (lectura de embeddings)

Tool description

Aquí puedes tomar los documentos cargados como base de información

Memory key

vector_store_key (debe coincidir con el de insert)

Fuente de embeddings

Embeddings OpenAI (compartido)

Consejo: El modo 'retrieve-as-tool' expone el vector store como una herramienta que el agente puede decidir usar o no. La tool description es clave: es lo que el LLM lee para saber cuándo usarla. Cuanto más descriptiva, mejor el agente decide cuando recuperar información.

System Prompt del Asistente Legal

El system prompt define el rol, las capacidades y el comportamiento del agente. Esta diseñado específicamente para uso legal con tono formal y citas obligatorias:

 # Rol: Asistente Legal

Descripcion

Eres un asistente legal para ayudar a un abogado a consultar documentos legales internos. Respuestas precisas, claras y fundamentadas. Tono profesional y formal. Respuestas cortas y puntuales. Si no conoces la respuesta, indica que no dispones de la informacion.

Capacidades

- Busqueda y recuperacion de informacion legal relevante.
- Respuestas concisas con fundamento en los documentos.
- Inclusion de referencia del documento consultado.
- Identificacion de ausencia de informacion.
- Confidencialidad y precision en todas las respuestas.

Comportamiento

1. Recepcion de la consulta del abogado.
2. Busqueda en la base de conocimiento.
3. Si hay informacion: resumen conciso + referencia (nombre de archivo, clausula).
   Si no hay informacion: 'No puedo encontrar la respuesta en los recursos disponibles.'
4. Confidencialidad total.
5. Tono profesional, formal, claro y directo.

Orden de uso

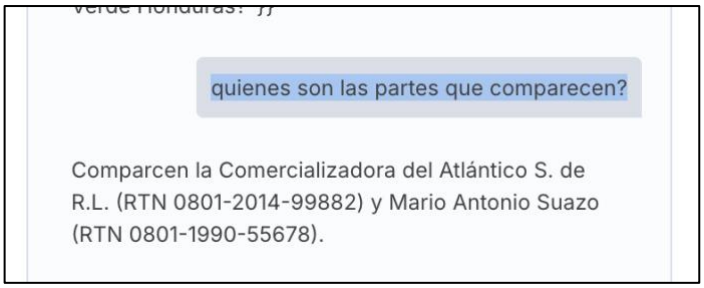
. Primero ejecuta la fase 1: sube los PDFs usando el formulario del Form Trigger

. Espera a que la ejecución de ingestión complete (barra de progreso en n8n)

. Luego usa el chat para hacer preguntas sobre los documentos cargados

. Si reinicias n8n, debes volver a cargar los documentos (vector store en memoria es volátil)

- Ollama NO es un buen modelo para responder y para usar RAG simplemente ejecuta el workflow y mira como intenta responder, aunque NO logra mirar dentro del documento correctamente. Sí que da alguna respuesta.
- Ejemplo de pregunta: ¿quiénes son las partes que comparecen?



Comparativa con los otros workflows

Aspecto	Pokémon Scraper	Wikipedia Agente	Sistema RAG
Patrón	Pipeline ETL	Agente reactivo	RAG (2 fases)
Fuente de datos	API externo publica	Wikipedia en tiempo real	Documentos PDF propios
Uso de IA	Sin IA	LLM + tool search	LLM + embeddings + vector DB
Conocimiento	No aplica	Conocimiento del modelo	Solo docs cargados
Memoria	Sin estado	Buffer 10 mensajes	Buffer 10 mensajes
Persistencia	Google Sheets	En memoria	En memoria (volátil)
Trigger	Manual	Chat webhook	Form + Chat webhook